File Handling in C

Sequential Access, Modes (w, r, a), CSV Parsing, and Memory-Efficient Stream Reading

Topic Overview & C-to-C++ Stream Mapping

Unlike object-oriented streams in C++ (ifstream, ofstream), standard C handles files using pointers to the FILE structure via the fopen(), fprintf(), fgets(), and fclose() interfaces.

- **Writing (ofstream → "w" mode):** Opens a file for writing. If the file already exists, its contents are erased. If it does not exist, it is created.
- **Reading (ifstream → "r" mode):** Opens an existing file for reading. Returns NULL if the file does not exist.
- **Appending (ofstream append → "a" mode):** Opens a file for writing at the end without destroying existing data.

Task Solutions

Task 1: Writing Songs to File ("w" Mode)

```c
#include <stdio.h>

int main() {
    FILE *fp = fopen("my_fav_songs.txt", "w");
    if (fp == NULL) {
        printf("Error: Could not create file!\n");
        return 1;
    }

    fprintf(fp, "Believer\n");
    fprintf(fp, "Blinding Lights\n");
    fprintf(fp, "Shape of You\n");
    fprintf(fp, "Kesariya\n");
    fprintf(fp, "Counting Stars\n");

    fclose(fp);
    printf("5 songs written successfully to my_fav_songs.txt\n");
    return 0;
}
```

Task 2: Reading Songs Line-by-Line ("r" Mode)

```c
#include <stdio.h>

int main() {
    FILE *fp = fopen("my_fav_songs.txt", "r");
```

```
    if (fp == NULL) {
        printf("Error: File not found!\n");
        return 1;
    }

    char song[100];
    int count = 1;
    while (fgets(song, sizeof(song), fp) != NULL) {
        printf("%d. %s", count++, song);
    }

    fclose(fp);
    return 0;
}
```

Task 3: Appending Data ("a" Mode)

```
#include <stdio.h>
#include <string.h>

int main() {
    char newSong[100] = "Perfect";
    FILE *fp = fopen("my_fav_songs.txt", "a");
    if (fp == NULL) return 1;

    fprintf(fp, "%s\n", newSong);
    fclose(fp);
    printf("Appended \"%s\" successfully.\n", newSong);
    return 0;
}
```

Task 4: Flipkart-Style Wishlist Tracker (CSV Parsing)

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main() {
    /* Writing mock wishlist */
    FILE *fp = fopen("wishlist.txt", "w");
    fprintf(fp, "Wireless Mouse,599.00\n");
    fprintf(fp, "Bluetooth Headphones,1499.00\n");
    fprintf(fp, "Phone Case,299.00\n");
    fclose(fp);

    /* Reading and parsing CSV wishlist */
    fp = fopen("wishlist.txt", "r");
    char line[120], name[50], priceStr[20];
    float total = 0;
    int count = 1;

    while (fgets(line, sizeof(line), fp) != NULL) {
        char *comma = strchr(line, ',');
        if (comma != NULL) {
```

```
            *comma = '\0';
            strcpy(name, line);
            strcpy(priceStr, comma + 1);
            float p = atof(priceStr);
            total += p;
            printf("%d. %-20s Rs.%s", count++, name, priceStr);
        }
    }
    printf("--------------------------\n");
    printf("Total Wishlist Value: Rs.%.2f\n", total);
    fclose(fp);
    return 0;
}
```

Task 5: Constant-Memory Line Counting

```
#include <stdio.h>

int main() {
    FILE *fp = fopen("insta_followers.txt", "r");
    if (fp == NULL) return 1;

    char username[100];
    int count = 0;
    while (fgets(username, sizeof(username), fp) != NULL) {
        if (username[0] != '\n' && username[0] != '\0') {
            count++;
        }
    }
    fclose(fp);
    printf("Total followers: %d\n", count);
    return 0;
}
```

Summary of File Operations

#	Operation	C Mode	C++ Stream Equivalent
1	Create & Write	`"w"`	`ofstream`
2	Read	`"r"`	`ifstream`
3	Append	`"a"`	`ofstream` (app mode)
4	CSV Write/Read	`"w"` then `"r"`	`ofstream` + `ifstream`

Key Takeaway: Always verify file pointers against `NULL`, ensure streams are closed with `fclose()`, and use safe bounded readers like `fgets()` to avoid buffer overflows.